DAY 3 — Network Automation, APIs, Orchestration and Infrastructure as Code

Beginner-friendly mental model

A modern network-automation platform is not:

> “A Python script that logs into 500 routers and runs commands.”

It is much closer to a **distributed infrastructure-control platform**.

A mature system must know:

- what infrastructure exists,
- what state it should be in,
- what state it is actually in,
- who is allowed to change it,
- whether a proposed change is safe,
- how to roll the change out gradually,
- how to recover from failures,
- whether the final state is correct,
- and exactly who changed what and why.

The most important conceptual shift is:

```
Old model
---------

Engineer
   |
   v
CLI commands
   |
   v
Device


Modern model
------------

User / System
      |
      v
Automation API
      |
      v
Policy + Workflow + Desired State
      |
      v
Safe Execution Platform
      |
      v
Controllers / Device APIs
      |
      v
Network
      |
      v
Telemetry
      |
      +------> Verification / Reconciliation
```

The platform therefore combines ideas from:

- networking,
- backend engineering,
- distributed systems,
- Kubernetes controllers,
- CI/CD,
- Infrastructure as Code,
- databases,
- workflow engines,
- security systems,
- and observability platforms.

1. Evolution of network automation

Network automation evolved because each previous approach solved one problem but exposed the next scaling problem.

Stage 1 — Manual CLI

An engineer logs into a device and executes commands.

```
Engineer
   |
 SSH
   |
Router
```

Example:

```
configure VLAN
configure interface
configure BGP peer
save configuration
```

Problem it solves

It allows humans to configure programmable network devices.

Limitation

At 5 devices, it is manageable.

At 5,000 devices, it becomes dangerous.

Humans introduce:

- typos,
- inconsistency,
- undocumented changes,
- forgotten steps,
- slow execution,
- configuration drift.

Stage 2 — Scripts

Instead of manually logging into devices, a program does it.

```
Python script
     |
     +---- Router 1
     +---- Router 2
     +---- Router 3
```

For example:

```
for device in devices:
    connect(device)
    execute(commands)
```

What this solves

- repetitive work,
- speed,
- consistency,
- bulk operations.

New problems

Scripts frequently become:

- tightly coupled to device CLI,
- difficult to retry,
- difficult to audit,
- dangerous during partial failures,
- hard to coordinate,
- hard to test.

The problem changes from:

> "How do I automate commands?"

to:

> "How do I safely operate distributed infrastructure?"

Stage 3 — Reusable automation

Automation is separated into reusable components.

For example:

```
get_device_state()
configure_vlan()
configure_bgp()
validate_routes()
backup_configuration()
```

Now common operations become libraries or modules.

Improvement

You gain:

- reuse,
- testing,
- abstraction,
- vendor adapters,
- standard error handling.

But something is still missing.

A function can configure a VLAN.

Who decides:

- whether the VLAN should exist?
- which devices receive it?
- whether the user is authorized?
- what happens if device 17 of 50 fails?

That requires orchestration.

Stage 4 — Workflow orchestration

Individual automation operations become steps inside controlled workflows.

```
Request
   |
Validate
   |
Approve
   |
Pre-check
   |
Execute
   |
Verify
   |
Audit
```

A workflow engine manages:

- state,
- dependencies,
- retries,
- timeouts,
- pause/resume,
- rollback.

This is the point where automation starts becoming a **platform**.

Stage 5 — Declarative automation

Instead of saying:

> Execute these commands.

you describe:

> This is the state I want.

Example:

```
Desired state:

VLAN 100
Name: payments
Sites: BLR1, BLR2
Gateway: 10.20.100.1/24
```

The system determines what operations are required.

This separates:

```
WHAT should exist
```

from:

```
HOW devices are configured
```

This is a fundamental improvement.

Stage 6 — Policy / intent-driven automation

The user expresses higher-level intent.

For example:

```
Connect payment-service
to database-service
with redundancy
and deny all other traffic.
```

The platform determines:

- VLANs,
- routing,
- VRFs,
- ACLs,
- firewall policy,
- device changes.

The abstraction moves higher.

Stage 7 — Closed-loop automation

The system continuously observes the infrastructure.

```
Desired state
     |
     v
Apply change
     |
     v
Observe network
     |
     v
Detect deviation
     |
     v
Correct state
```

Humans are no longer required for every corrective action.

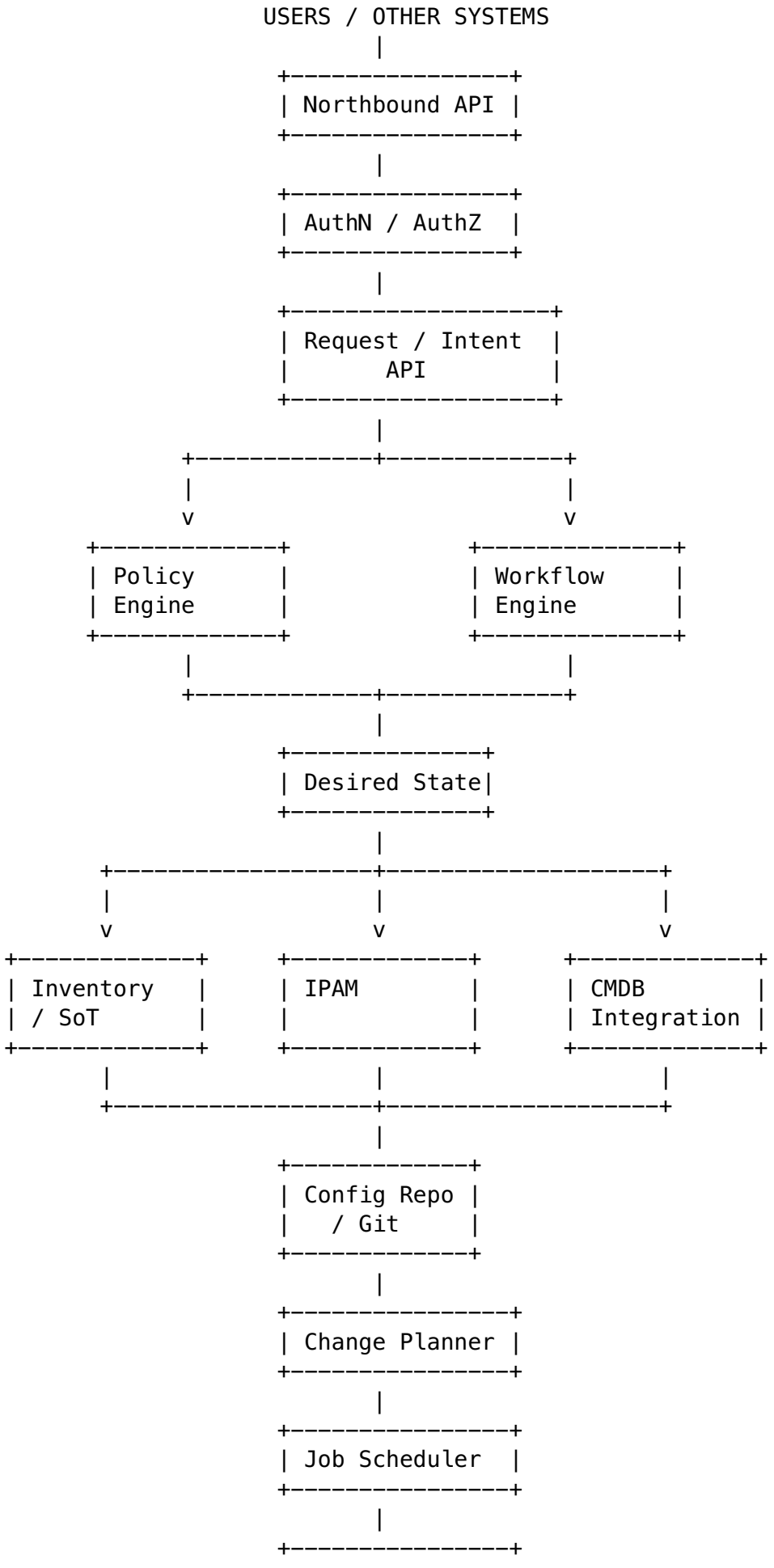
This is powerful—but dangerous if implemented badly.

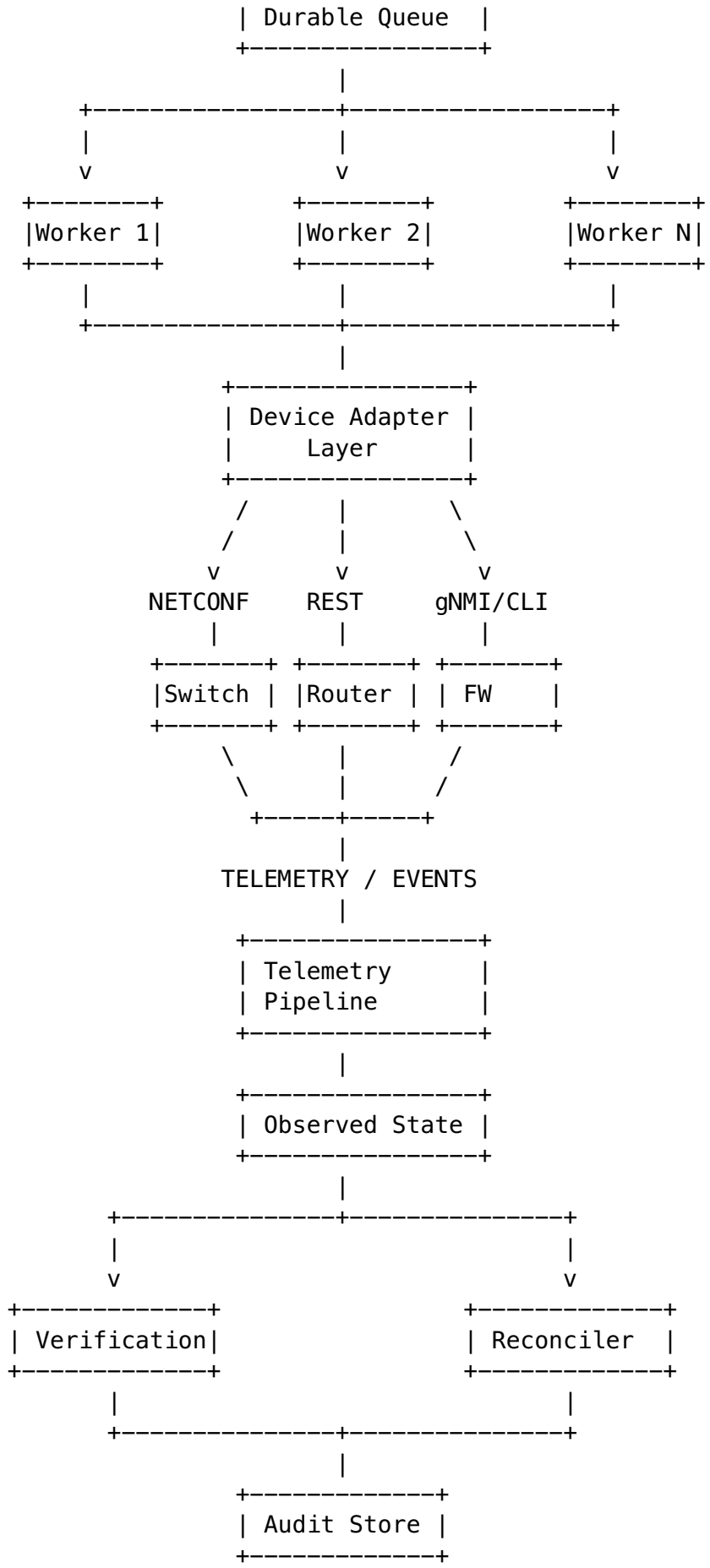
A faulty controller can repair thousands of devices incorrectly just as efficiently as it can repair them correctly.

---

2. Core automation architecture

A mature network-automation platform might look like this:


USERS / OTHER SYSTEMS
Northbound API
AuthN / AuthZ
Request / Intent API
Policy Engine
Workflow Engine
Desired State
Inventory / SoT
IPAM
CMDB Integration
Config Repo / Git
Change Planner
Job Scheduler



There are effectively **three planes** here.

Intent plane

Determines what should happen.

Includes:

- APIs,
- source of truth,
- desired state,
- policy.

Execution plane

Makes the change.

Includes:

- workflows,
- queues,
- workers,
- adapters,
- controllers.

Observation plane

Determines what actually happened.

Includes:

- telemetry,
- verification,
- reconciliation,
- audit.

Separating these concerns makes the system much safer.

---

3. Inventory, IPAM, CMDB and source of truth

Automation cannot safely control infrastructure it does not understand.

Suppose an automation request says:

> Deploy VLAN 200 across production leaf switches in Bangalore.

The system must answer:

```
Which site is Bangalore?
Which switches are leaf switches?
Which are production?
What software versions do they run?
Which interfaces exist?
Does VLAN 200 already exist?
What IP ranges are allocated?
Who owns the service?
```

That information comes from inventory and related systems.

Device inventory

Typically records:

```
device_id
hostname
vendor
model
role
site
management_ip
OS
software_version
serial_number
status
```

Example:

```
hostname: blr-leaf-21
vendor: Arista
role: leaf
site: BLR-DC1
software: EOS 4.x
```

Sites

The system needs topology and organizational context.

```
Region
 └── Bangalore
      ├── DC1
      ├── DC2
      └── Branch-17
```

Site information may include:

- physical location,
- failure domain,
- timezone,
- maintenance window,
- network role.

Interfaces

Example record:

```
Device: blr-leaf-21
Interface: Ethernet5
Speed: 100G
Peer: blr-spine-02
Operational state: up
```

Interface knowledge becomes important when evaluating topology impact.

IPAM

IP Address Management tracks:

```
VRF
Network
Subnet
IP address
Gateway
Allocation
Owner
Status
```

Example:

```
10.20.0.0/16       Bangalore production

10.20.100.0/24     Payments VLAN

10.20.100.10        payment-service-1
```

Without IPAM automation, two systems might accidentally allocate the same subnet.

CMDB

A Configuration Management Database often represents the broader business environment.

For example:

```
Application:
payment-platform

depends on:
network-segment-100
load-balancer-12
database-cluster-4
```

A network automation platform may use CMDB data to understand application impact.

Device capabilities

Not every device supports the same features.

For example:

```
Device A:
EVPN supported
gNMI supported
NETCONF supported

Device B:
EVPN unsupported
gNMI unsupported
CLI only
```

Automation must account for this.

Topology

Topology describes relationships.

```
        Spine1
       /      \
    Leaf1     Leaf2
      |         |
    ServerA   ServerB
```

Changing `Spine1` could affect both leaves.

Topology-aware automation therefore has a safer view of blast radius.

Configuration intent

Inventory says:

> what exists.

Intent says:

> what should exist.

Example:

```
Site: BLR1

Desired VLANs:
100 payments
200 analytics

Desired routing:
BGP ASN 65001

Desired uplinks:
2 redundant spine links
```

What makes a system authoritative?

Suppose VLAN ownership exists in:

```
Spreadsheet
NetBox
ServiceNow
Git repository
Device configuration
```

Which one is correct?

A mature organization explicitly defines authority.

For example:

```
IP allocations       -> IPAM
Device ownership     -> CMDB
Network intent       -> Git
Operational state    -> devices / telemetry
```

The key principle is:

> Every data domain should have a clearly defined authoritative source.

Duplicate sources create risk

Imagine:

```
Git:
VLAN 100 = payments

CMDB:
VLAN 100 = finance

Device:
VLAN 100 = legacy-app
```

Automation cannot safely proceed without resolving the inconsistency.

Synchronization

Sometimes multiple systems must contain the same data.

Therefore synchronization processes may exist:

```
CMDB
  |
  v
Network inventory

IPAM
  |
  v
Automation platform
```

However synchronization should preserve authority.

You do not want accidental circular updates:

```
System A
  |
  v
System B
  |
  v
System A
```

because conflicting updates can endlessly overwrite one another.

---

4. Desired state and actual state

This idea is central to modern automation.

Desired state

What infrastructure is supposed to look like.

Example:

```
VLAN 100 must exist
on leaf1, leaf2, leaf3.
```

Observed state

What infrastructure currently looks like.

```
leaf1 -> VLAN 100 exists
leaf2 -> VLAN 100 exists
leaf3 -> VLAN 100 missing
```

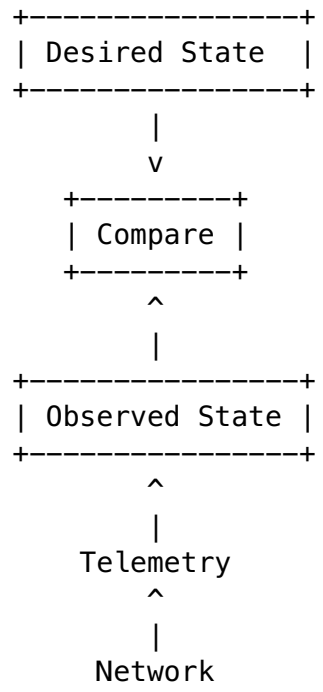
The difference is called **drift**.

Reconciliation

The system compares:

```
desired state
      vs
observed state
```

and determines what must change.



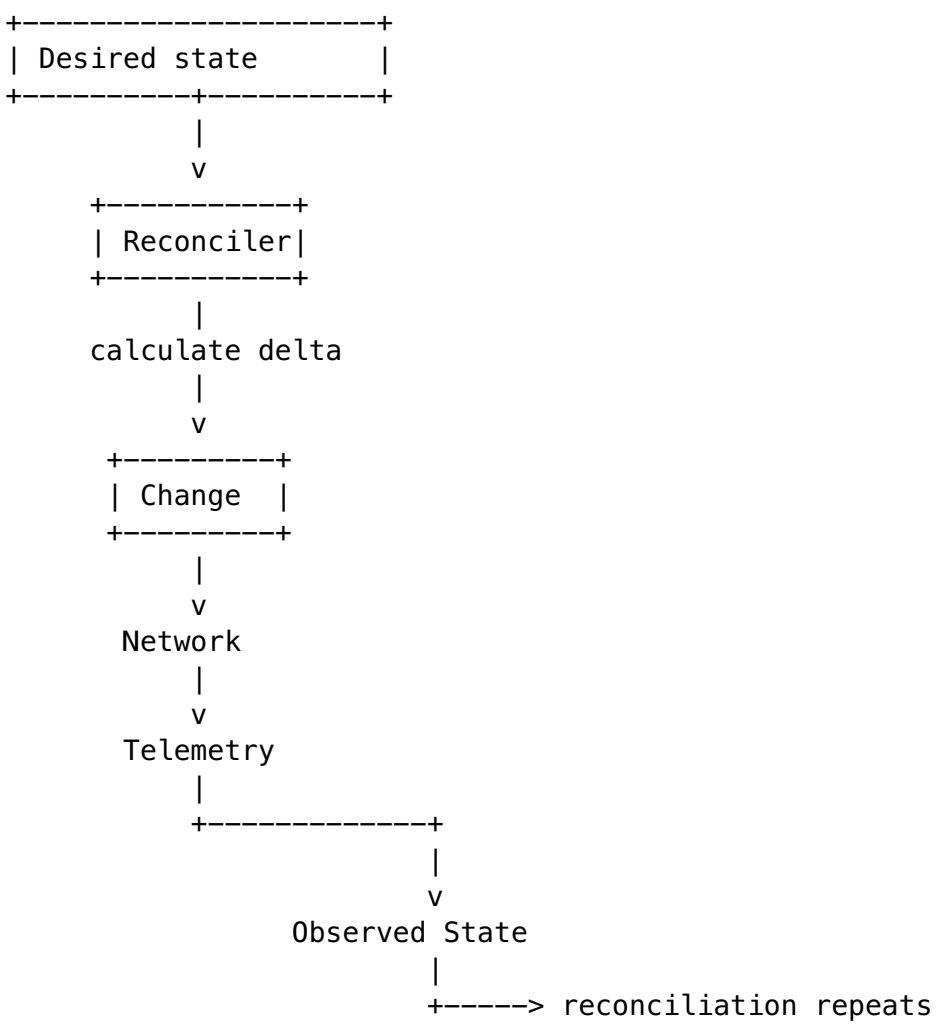
If they differ:

```
Desired
VLAN 100 exists

Observed
VLAN 100 missing
```

the reconciler calculates:

```
Action:
ensure VLAN 100 exists
```

Then:



This is very similar to Kubernetes.

You create:

```
replicas: 3
```

Kubernetes does not interpret this as:

> Run exactly one command creating three Pods.

It interprets it as:

> The desired system state should continuously contain three replicas.

If a Pod disappears, Kubernetes acts again.

Network controllers increasingly use the same model.

---

5. Idempotency

Idempotency means:

> Performing the same intended operation multiple times produces the same final result as performing it once.

Consider two operations.

Imperative

```
Create VLAN 100
```

Run once:

```
VLAN created.
```

Run again:

```
ERROR: VLAN already exists.
```

Possibly worse, poorly designed systems might create duplicate resources.

Idempotent intent

```
Ensure VLAN 100 exists.
```

The system behaves like this:

```
if VLAN 100 missing:
    create it

if VLAN 100 already correct:
    do nothing

if VLAN 100 exists incorrectly:
    reconcile it
```

The operation can safely run repeatedly.

Why idempotency is extremely important

Suppose an automation platform sends:

```
Configure VLAN 100
```

The device processes it successfully.

But the worker crashes before recording success.

The workflow engine sees:

```
step status = unknown
```

What should it do?

Retry.

Without idempotency:

```
Retry might duplicate or corrupt state.
```

With idempotency:

```
Retry safely converges on desired state.
```

Therefore idempotency makes retries practical.

It is important for:

- network failures,
- worker crashes,
- workflow restarts,
- duplicate API requests,
- partial failures,
- recovery.

---

6. Network-management interfaces

The automation platform needs ways to interact with devices.

Interface	Typical role	Structured?	Configuration	Telemetry
SSH/CLI	Human/device command interface	Mostly no	Yes	Limited
SNMP	Monitoring/management	Partly	Limited	Strong historically
NETCONF	Model-driven config	Yes	Strong	Yes
RESTCONF	HTTP access to YANG data	Yes	Strong	Yes
REST APIs	Vendor/controller APIs	Yes	Strong	Strong
gNMI	Modern model-driven RPC/streaming	Yes	Yes	Excellent

SSH / CLI

Example interaction:

```
ssh router1

configure terminal
router bgp 65001
neighbor ...
```

Strength

- nearly universally available,
- exposes vendor capabilities quickly.

Weakness

CLI is fundamentally designed for humans.

Output might look like:

```
Interface Ethernet1 is up, line protocol is up
```

Automation must parse text.

A software upgrade may change wording:

```
Ethernet1 state UP, protocol UP
```

and the parser breaks.

Therefore CLI automation tends to be fragile.

SNMP

SNMP became widely used for monitoring.

It exposes structured identifiers called OIDs.

Typical use:

```
CPU
interface counters
device state
errors
```

Historically it has been much stronger for telemetry than configuration.

Strengths:

- widely supported,
- standardized monitoring.

Limitations:

- cumbersome data model,
- configuration workflows are awkward,
- not ideal for rich modern streaming telemetry.

NETCONF

NETCONF was designed for structured network configuration.

Instead of sending CLI text, a client manipulates configuration data represented structurally.

Example conceptual operation:

```
edit-config
```

against structured configuration.

Major advantages:

- machine-readable,

- transactional capabilities on some devices,
- validation,
- standardized semantics.

NETCONF commonly uses **YANG models** to describe data.

RESTCONF

RESTCONF exposes similar YANG-modeled network data using HTTP-style operations.

Conceptually:

```
GET
POST
PATCH
DELETE
```

against modeled network resources.

It is attractive to developers already familiar with REST APIs.

REST APIs

Many vendors and network controllers expose custom REST APIs.

Example:

```
POST /networks
GET /devices
PATCH /policies/123
```

Strength:

- easy integration with software platforms.

Limitation:

- API semantics and data models vary between vendors.

---

gNMI

gNMI is a modern RPC interface commonly associated with:

- configuration,
- state retrieval,
- streaming telemetry.

It is especially powerful for subscriptions.

Instead of:

```
poll every 30 seconds
```

you can request:

```
send updates when this state changes
```

This supports near-real-time observability.

---

7. Model-driven networking

Imagine automation receiving this:

```
"interface xe0 has speed high"
```

What does `high` mean?

1 Gbps?

10 Gbps?

100 Gbps?

Machines need precise schemas.

YANG

YANG is a modeling language used to describe network configuration and operational data.

Conceptually a model may specify:

```
interface
 ├── name: string
 ├── enabled: boolean
 ├── mtu: integer
 └── address
      ├── ip
      └── prefix-length
```

Now systems understand:

- valid fields,
- types,
- hierarchy,
- constraints.

Schema validation

Suppose MTU must be:

```
1280–9216
```

An automation request says:

```
MTU = 50
```

A schema-aware system can reject the request before reaching the device.

This significantly improves safety.

YANG and NETCONF

A useful conceptual relationship is:

```
YANG
  |
  | describes data
  v
Network data model
  ^
  |
NETCONF
  |
transports / manipulates the modeled data
```

YANG defines the structure.

NETCONF provides the management protocol.

YANG and RESTCONF

Similarly:

```
YANG
  |
defines resources
  |
RESTCONF
  |
accesses resources via HTTP semantics
```

OpenConfig

Vendors historically exposed different models.

For example:

```
Cisco interface model
Juniper interface model
Arista interface model
```

Automation must understand each one.

OpenConfig attempts to provide vendor-neutral models.

Conceptually:

```
Automation
      |
      v
OpenConfig model
      |
  +---+---+
  |   |   |
Cisco Juniper Arista
```

The same high-level schema can therefore work across multiple platforms.

Real environments still require vendor-specific handling for features outside standardized models.

OpenConfig + gNMI

A common modern combination is:

```
OpenConfig
   =
data model

gNMI
   =
mechanism for accessing/subscribing to that data
```

For example:

```
Subscribe:
interfaces/interface/state/counters
```

The device streams updates.

Device capability models

Automation must understand what individual devices support.

Example:

```
Device A supports:
BGP
EVPN
OpenConfig
gNMI

Device B supports:
BGP
no EVPN
partial OpenConfig
NETCONF
```

The workflow should validate compatibility before generating a change.

8. Configuration templating

Templates reduce duplicated configuration.

Instead of writing:

```
interface Ethernet1
description uplink-spine1
ip address 10.0.1.1
```

for every device, we might use:

```
interface {{ interface }}
description {{ description }}
ip address {{ ip_address }}
```

and provide variables.

```
interface = Ethernet1
description = uplink-spine1
ip_address = 10.0.1.1
```

Reusable configuration

The same template can generate:

```
BLR
HYD
MUM
DEL
```

configurations using site-specific variables.

Template risks

Templates are not automatically safe.

Imagine:

```
MTU = {{ site_mtu }}
```

An incorrect assumption could push an invalid MTU to hundreds of interfaces.

Risks include:

Invalid assumptions

The template assumes all devices have:

```
Ethernet1
```

but some use:

```
xe-0/0/1
```

Vendor differences

Cisco syntax may not work on Juniper.

Template drift

Multiple copies evolve independently.

```
template-v1
template-v1-final
template-v1-final2
template-new
```

Now nobody knows the authoritative one.

Insufficient validation

A template renders successfully but creates logically dangerous configuration.

Therefore mature systems perform:

```
template rendering
        |
        v
schema validation
        |
        v
policy validation
        |
        v
device-specific validation
```

---

9. Infrastructure as Code

Infrastructure as Code means infrastructure state is defined as machine-readable artifacts rather than undocumented manual changes.

Core principles include:

```
Declarative definitions
        +
Version control
        +
Code review
        +
Automated validation
        +
Reproducible execution
```

Example

Instead of manually creating:

```
VRF payments
VLAN 100
subnet 10.20.100.0/24
```

we store something conceptually like:

```
network:
  name: payments
  vrf: payments
  vlan: 100
  subnet: 10.20.100.0/24
```

Now Git preserves history.

```
Commit A
Created VLAN

Commit B
Changed subnet

Commit C
Expanded to BLR2
```

Why this matters

Manual infrastructure has poor reproducibility.

Suppose production works.

Can you recreate it after a disaster?

With IaC:

```
desired infrastructure
        |
      code
        |
      Git
        |
automation platform
        |
infrastructure
```

The desired configuration is reproducible.

Networking and IaC

Networking makes IaC more difficult because network state often includes:

- mutable existing hardware,
- vendor-specific features,
- physical topology,
- transient routing state,
- long-running changes.

Nevertheless the principles remain extremely valuable.

---

10. GitOps

GitOps takes IaC further.

Git becomes the authoritative desired-state source.

```
Engineer
   |
   v
Git branch
   |
Pull request
   |
Review
   |
Merge
   |
   v
Controller detects change
   |
   v
Infrastructure reconciled
```

The workflow becomes:

```
Change intent
     |
     v
Pull request
     |
validation
     |
approval
     |
merge
     |
reconciliation
```

Advantages

Reviewability

Changes receive peer review before deployment.

Auditability

Git records:

```
who
what
when
why
```

Rollback visibility

Previous desired states are available.

Reproducibility

Infrastructure definitions exist independently of device state.

Limitations for networking

GitOps is not perfect for every network operation.

Suppose a fiber link fails.

You probably do not want:

```
Open PR
Wait for reviewer
Merge PR
Repair routing
```

Operational events may require immediate controller actions.

Another problem:

Git represents intended state well but not rapidly changing operational state.

You would not normally commit:

```
Interface packet counter = 18293821891
```

every second.

Therefore a practical design is:

```
Git
 |
 | durable desired state
 v
Controller

Telemetry
 |
 | dynamic observed state
 v
Controller
```

Git does not need to contain every operational observation.

---

11. Policy as Code

Policy as Code expresses governance rules in machine-evaluable form.

Example rules:

```
Production BGP changes require approval.

No management interface may be internet-exposed.

Every production switch must have two uplinks.

VLAN IDs 1–99 are reserved.

Changes affecting >20 devices require staged rollout.
```

The automation platform evaluates these automatically.

Network example

Requested change:

```
Deploy ACL allowing 0.0.0.0/0 -> management interface
```

Policy engine responds:

```
DENIED

Reason:
Management plane cannot be exposed to untrusted networks.
```

The protection exists regardless of which engineer submits the request.

Guardrails vs workflows

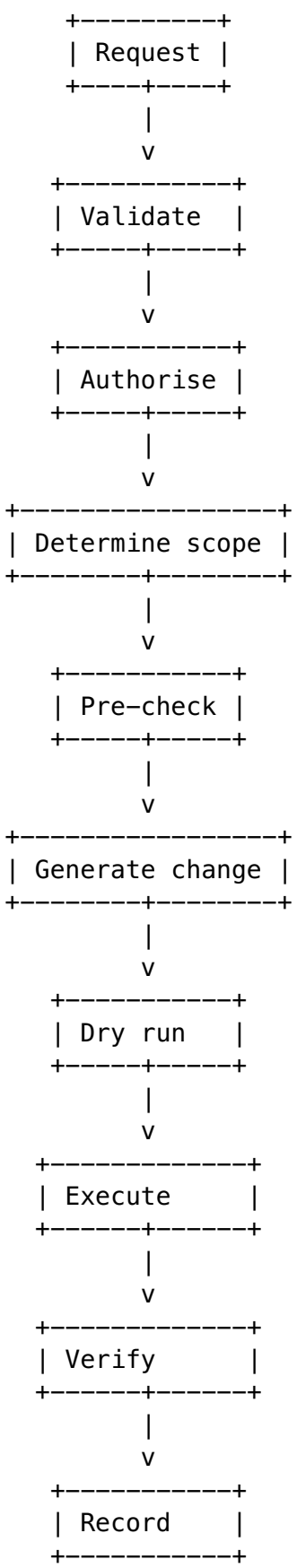
A useful distinction:

```
Workflow:
HOW the change proceeds

Policy:
WHETHER the change is permitted
```

12. Workflow orchestration

Consider deploying a new network segment.

A safe lifecycle might be:



Workflow

The entire business process.

Example:

```
Provision network for new application.
```

Step

A unit of execution.

Example:

```
Allocate subnet.
Configure switches.
Configure firewall.
Verify reachability.
```

Dependency

Some steps cannot start until others succeed.

```
Allocate subnet
      |
      v
Configure VLAN
      |
      v
Configure routing
```

Routing cannot be configured correctly without knowing the allocated subnet.

State

The workflow engine remembers where the workflow is.

```
Step 1 succeeded
Step 2 succeeded
Step 3 running
Step 4 pending
```

This is critical during crashes.

Retry

If a transient error occurs:

```
device temporarily unreachable
```

the step may retry.

Compensation

If a later step fails, the system may undo previous work.

Example:

```
Allocated VLAN
Configured leafs
Firewall configuration failed
```

Possible compensating action:

```
Remove new VLAN from leafs
Release allocation
```

Pause/resume

A network workflow might pause for:

- approval,
- maintenance window,
- incident investigation,
- failed validation.

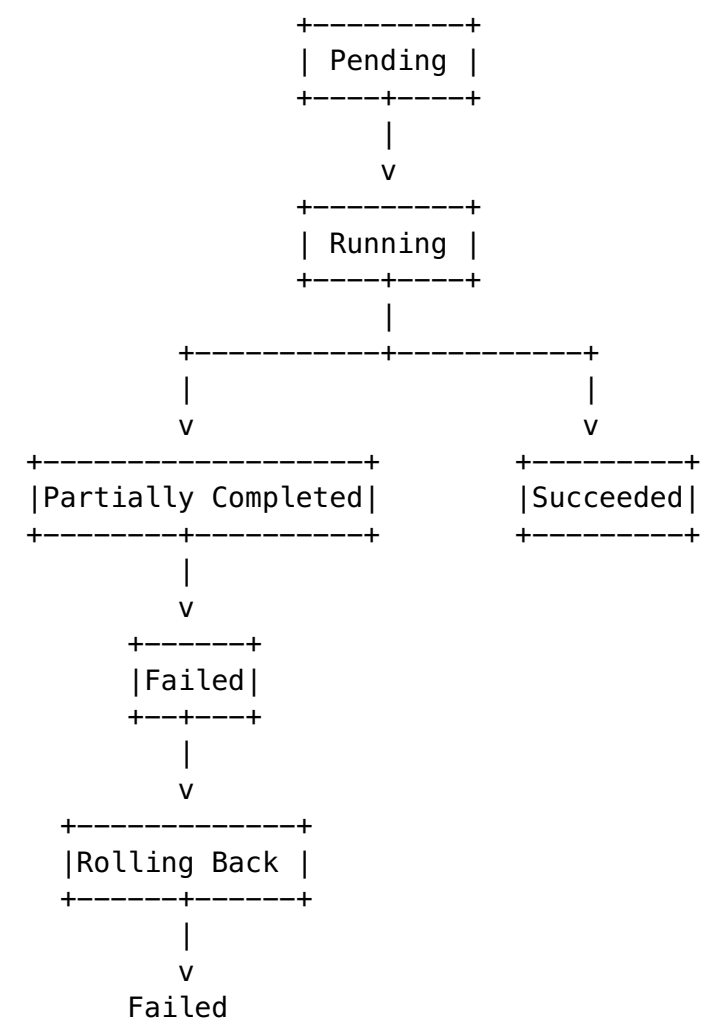
It must later resume without losing state.

---

13. Workflow state machines

Long-running infrastructure workflows require explicit states.

Example:



Cancellation may appear from multiple states:

```
Pending -> Cancelled

Running -> Cancelling -> Cancelled
```

Why explicit states matter

Without state modeling, an operator might only see:

```
job failed
```

But these are radically different:

```
0/100 devices changed

vs

98/100 devices changed
```

The appropriate recovery is completely different.

A state machine captures this.

---

14. Asynchronous job execution

Suppose an API request asks:

```
Upgrade 500 switches.
```

That might take 40 minutes.

You do not want:

```
HTTP request stays open for 40 minutes.
```

Several things could break:

- reverse-proxy timeout,
- client disconnect,
- application restart,
- network interruption.

Instead:

```
POST /changes
```

might respond:

```
202 Accepted

job_id = 71ac92
```

Then:

```
GET /jobs/71ac92
```

returns:

```
status: RUNNING
progress: 150/500
```

Architecture

```
Client
  |
POST request
  |
  v
API
  |
create job
  |
  v
Job DB
  |
  v
Queue
  |
  v
Worker
  |
  v
Network
```

The API and execution lifecycle become decoupled.

Polling

Client periodically asks:

```
GET /jobs/123
```

Events / callbacks

Instead of polling:

```
Job finished
     |
     v
Event broker
     |
     v
Interested system
```

or:

```
Webhook callback
```

can notify the caller.

---

15. Queues and backpressure

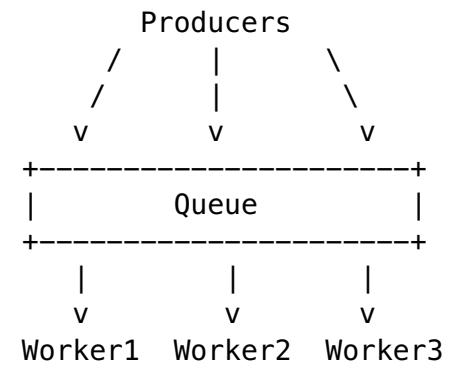
A queue decouples:

```
request arrival
```

from:

```
request execution
```

Architecture:



Producer

Creates jobs.

For example:

```
API
scheduler
event processor
```

Consumer

Processes jobs.

Usually:

```
worker processes
```

Queue depth

Number of waiting jobs.

Suppose:

```
Incoming:
100 jobs/minute

Processing:
30 jobs/minute
```

Queue growth becomes:

```
+70 jobs/minute
```

After 10 minutes:

```
700 jobs waiting
```

Eventually latency becomes unacceptable.

Backpressure

Backpressure tells upstream producers:

> The system cannot safely accept unlimited additional work.

Possible techniques:

```
rate limit
reject requests
delay producers
prioritize workloads
increase workers
reduce concurrency per device
```

Without backpressure, a busy system can collapse itself.

For example:

```
Queue grows
   |
more workers launched
   |
more SSH connections
   |
device CPUs overloaded
   |
requests slower
   |
workers retry
   |
more load
```

This is a feedback loop toward failure.

---

16. Failure handling

A mature platform distinguishes failures rather than treating all errors equally.

Retryable failure

Example:

```
TCP connection temporarily failed
```

Retrying later may work.

Permanent failure

Example:

```
Device does not support EVPN.
```

Retrying 100 times will not help.

The change must be corrected.

Timeout

The system cannot wait forever.

Example:

```
NETCONF operation expected <30 sec
```

After the timeout, the operation enters an uncertain state.

The platform may need to query actual device state before deciding whether to retry.

Device unavailable

Could result from:

- reboot,
- network isolation,
- management-plane failure.

The workflow may retry or pause.

Partial success

Example:

```
98 devices succeeded
2 devices failed
```

This is often the most important infrastructure failure mode.

A binary success/failure model is insufficient.

Worker crash

A worker may die after a device accepted the change.

The new worker must determine:

```
Did the previous operation succeed?
```

This is why:

- idempotency,
- operation IDs,
- observed-state checks

are so important.

Exponential backoff

Instead of:

```
retry every 1 second forever
```

use increasing delays:

```
1s
2s
4s
8s
16s
```

This reduces pressure on a failing dependency.

Jitter

If 10,000 workers all wait exactly 8 seconds, they all retry simultaneously.

This causes:

```
retry storm
```

Jitter adds randomness:

```
7.4 sec
8.2 sec
9.0 sec
6.7 sec
```

Requests spread over time.

Dead-letter handling

Jobs that repeatedly fail may move to a dead-letter queue.

```
main queue
    |
 repeated failure
    v
dead-letter queue
```

This prevents one bad job from retrying forever.

Operators can inspect it separately.

---

17. Exactly-once and deduplication

"Exactly once" sounds simple:

> Execute every operation once and only once.

Distributed systems make this surprisingly difficult.

Imagine:

```
Worker -> Router
```

Worker sends:

```
Create VLAN 100
```

Router performs the operation.

Then the network connection fails before the response reaches the worker.

Worker sees:

```
timeout
```

Did the device perform the operation?

The worker cannot know from the response.

If it retries, the operation may happen twice.

If it does not retry, the operation might not have happened at all.

Practical model

Many robust systems use:

```
at-least-once delivery
        +
idempotent processing
        +
deduplication
```

instead of trying to guarantee perfect exactly-once semantics.

Unique operation ID

Every logical change gets an identifier.

```
operation_id:
7a92b3
```

A duplicate request with the same ID can be recognized.

```
if operation_id already completed:
    return previous result
```

Example

User submits:

```
Provision VLAN 100

Idempotency-Key:
abc123
```

Client loses the response and retries.

The platform sees:

```
abc123 already processed
```

and returns the existing job instead of creating another change.

---

18. Concurrency and conflicting changes

Suppose two workflows run:

```
Workflow A:
Change interface Ethernet1 to VLAN 100.

Workflow B:
Change interface Ethernet1 to VLAN 200.
```

Both are individually valid.

Together they conflict.

Locks

Workflow A acquires:

```
lock:
device1/interface/Ethernet1
```

Workflow B waits.

Advantage

Simple correctness model.

Limitation

Locks reduce concurrency.

Poorly handled locks can also become stuck.

Leases

A lease is a lock with expiration.

```
lock owner: worker17
expires: 17:10:30
```

If worker17 crashes, the lock eventually expires.

Useful in distributed systems.

Optimistic concurrency

Instead of locking first:

1. Read current resource.
2. Remember version.
3. Prepare change.
4. Update only if version remains unchanged.

Example:

```
Interface version = 17

Change:
VLAN 100 -> VLAN 200

Update condition:
version must still be 17
```

If another workflow updates it:

```
version = 18
```

your write is rejected.

Compare-and-set

Conceptually:

```
Change value from X to Y

only if current value is still X.
```

This prevents overwriting unexpected changes.

Resource ownership

Some platforms enforce ownership boundaries.

For example:

```
Routing controller owns BGP config.

Security controller owns ACLs.

IPAM owns IP allocations.
```

Two systems should not independently mutate the same fields.

This dramatically reduces conflicting automation.

19. Transactions and compensation

Database transactions allow:

```
BEGIN

change A
change B
change C

COMMIT
```

If one fails:

```
ROLLBACK
```

and the database restores the previous state.

Networks usually cannot provide one global ACID transaction covering hundreds of independently managed devices.

Imagine:

```
Router1 configured
Router2 configured
Router3 unreachable
Router4 configured
```

There is no global database transaction manager capable of instantly restoring all hardware states.

Saga-style thinking

Distributed operations instead use a series of steps.

```
Step A
   |
Step B
   |
Step C
```

Each successful step may have a compensating action.

Example:

```
A: allocate VLAN
A': release VLAN

B: configure switches
B': remove switch config

C: configure firewall
C': remove firewall rule
```

If C fails:

```
run B'
run A'
```

Rollback versus forward recovery

Sometimes rollback is dangerous.

Suppose:

```
old firmware contains critical bug
```

During upgrade:

```
200 devices upgraded
10 failed
```

Returning 200 devices to vulnerable firmware may be worse.

The better strategy may be:

```
fix remaining 10
```

This is **forward recovery**.

Infrastructure platforms therefore need judgment about:

```
rollback
versus
continue toward desired state
```

---

20. Safe rollout

Never treat:

```
change 10,000 devices
```

as one giant undifferentiated operation.

Use progressive control.

Dry run

Calculate what would change without performing it.

Example:

```
Current:
VLAN 100 absent

Desired:
VLAN 100 present

Planned:
create VLAN 100
```

The operator can review the plan.

Pre-check

Check conditions before modifying state.

Example:

```
device reachable?
configuration clean?
software supported?
redundancy healthy?
```

Canary

Change a very small subset first.

```
1000 routers

Canary:
5 routers
```

Verify them.

Then continue.

Batch rollout

```
Batch 1: 5
Batch 2: 20
Batch 3: 100
Batch 4: remainder
```

Progressive rollout

Each batch depends on health signals.

```
Deploy batch
    |
    v
Observe
    |
healthy?
 /    \
yes   no
 |     |
next  pause
batch
```

Blast-radius limits

Rules could say:

```
Never change both redundant core routers simultaneously.

Never modify >10% of site capacity at once.
```

These are extremely valuable safety controls.

Maintenance windows

Some changes should only run during approved windows.

Example:

```
Saturday 01:00–04:00
```

The workflow engine may pause until the window begins.

Automated rollback

If health deteriorates beyond a threshold:

```
packet loss > 5%
BGP sessions drop > 10%
```

the rollout may stop or revert.

---

21. Validation

Automation should not equate:

> API returned success

with:

> Network change was successful.

Those are different statements.

Pre-change validation

Reachability

Can the automation platform contact the device?

Current state

Is the device already in an unexpected condition?

Example:

```
Expected:
BGP peers healthy

Actual:
30% already down
```

You probably should not start a risky change.

Device capability

Does the device support the feature?

Dependency state

Suppose changing router A relies on router B being available.

Check B first.

Policy compliance

Does the intended configuration violate guardrails?

Capacity

Example:

```
TCAM usage already 97%
```

Adding thousands of ACL entries could exhaust hardware capacity.

Post-change validation

After making the change, verify actual functionality.

Example VLAN deployment:

```
Did configuration appear?

Is interface state healthy?

Did spanning tree remain stable?

Are expected routes present?

Can application endpoints communicate?

Did packet loss increase?
```

Verification should increasingly operate at the **service level**, not merely the configuration level.

---

22. Topology and dependency awareness

Network infrastructure is interconnected.

Changing one device may affect many others.

```
            Core1
           /     \
        Spine1   Spine2
         /  \     /  \
      Leaf1 Leaf2 Leaf3 Leaf4
```

Changing `Core1` may affect every downstream device.

Routing peer impact

Suppose:

```
Router A ––– BGP ––– Router B
```

Changing Router A's BGP configuration impacts Router B.

A topology-aware system knows this dependency.

Redundant paths

Suppose:

```
Site
 | \
 |  \
R1   R2
 \   /
 Internet
```

Both provide redundancy.

Automation must not upgrade R1 and R2 simultaneously.

Topology allows the system to reason:

```
R1 and R2 share failure responsibility.
```

Therefore:

```
Upgrade R1
verify
Upgrade R2
```

is safer.

Application dependency

Network changes can also affect services.

```
Payment API
     |
Load balancer
     |
Firewall
     |
Leaf switch
     |
Database
```

Changing the firewall is therefore not just:

> a firewall change.

It may be:

> a payment-service dependency change.

---

23. Event-driven automation

Traditional automation often uses polling.

```
Every 60 sec:
check interface status
```

That means failures may be detected up to approximately one minute later.

Events

Instead, systems can emit:

```
interface down
BGP neighbor lost
device rebooted
```

immediately.

Webhooks

A system sends an HTTP notification:

```
Event source
    |
    v
POST /events
    |
Automation platform
```

Useful for system-to-system integrations.

Message brokers

For larger environments:

```
Network events
      |
      v
 Kafka / Event bus
      |
 +----+----+
 |         |
 v         v
Automation Observability
```

This decouples event producers from consumers.

Reactive automation

Example:

```
Telemetry detects link failure
        |
        v
Event generated
        |
        v
Automation validates topology
        |
        v
Traffic rerouted
        |
        v
Health verified
```

This is one path toward closed-loop automation.

---

24. Streaming telemetry

Traditional monitoring:

```
Monitoring server
      |
poll every 30 seconds
      |
      v
Device
```

Problems:

- many requests,
- delayed state,
- bursty polling,

- unnecessary requests when nothing changes.

Streaming model

```
Monitoring system
       |
 subscribe
       |
       v
Device
       |
continuous updates
       |
       v
Telemetry pipeline
```

Examples of streamed data:

```
interface counters
BGP state
CPU
memory
queue depth
optics
routing state
```

---

gNMI relationship

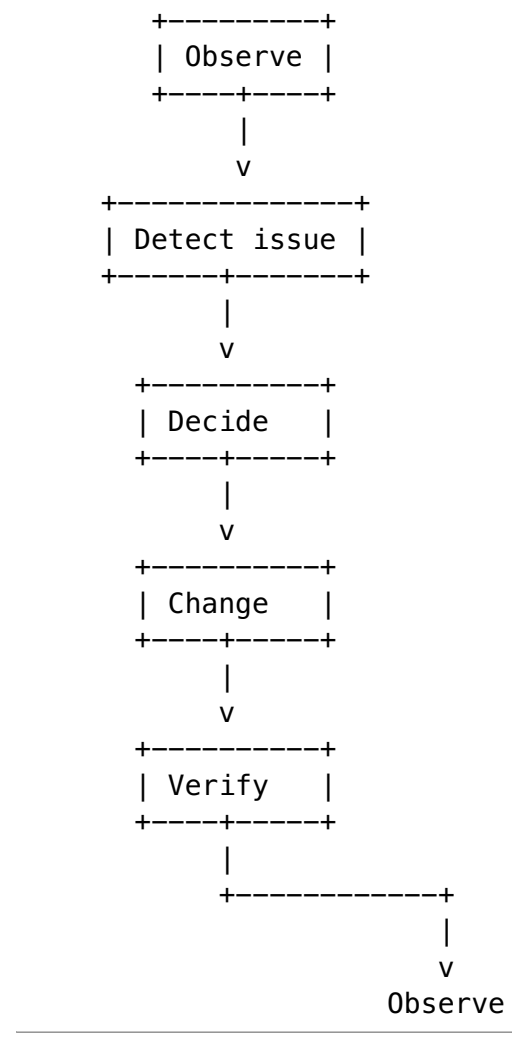
A common architecture is:

```
OpenConfig
     |
defines telemetry structure
     |
     v
gNMI
     |
subscription transport
     |
     v
Telemetry collector
```

---

25. Closed-loop automation

Closed-loop systems continuously compare infrastructure behaviour with intent.



Example:

```
Desired:
2 healthy BGP uplinks

Observed:
1 healthy

Controller:
detects failure

Policy:
automatic remediation allowed

Action:
activate alternate path

Verification:
2 healthy routes restored
```

Benefits

- faster remediation,

- reduced operator workload,
- consistent repair,
- continuous drift correction.

Risks

Automation multiplies both correctness and mistakes.

A bad detection rule might trigger thousands of unnecessary changes.

A safe closed-loop system therefore needs:

```
confidence thresholds
blast-radius limits
rate limits
policy
verification
audit
human escalation
```

Some actions may be fully autonomous.

Others should remain:

```
detect automatically
recommend automatically
human approves
```

---

26. API architecture for automation

A network automation platform usually exposes APIs to:

- humans,
- portals,
- CI/CD,
- other infrastructure platforms.

Resource-oriented APIs

Examples:

```
GET /devices
GET /sites
GET /jobs/123
POST /changes
GET /changes/456
```

Resources should have stable identities.

Synchronous APIs

Good for quick operations.

Example:

```
GET /devices/router1
```

Response arrives immediately.

Asynchronous APIs

Good for long-running changes.

```
POST /changes
```

Response:

```
202 Accepted

job_id: abc123
```

Job endpoints

Useful API pattern:

```
POST   /jobs
GET    /jobs/{id}
POST   /jobs/{id}/cancel
GET    /jobs/{id}/events
```

Idempotency keys

Clients can send:

```
Idempotency-Key: request-782
```

to prevent accidental duplicate changes.

Pagination

Do not return 100,000 devices in one response.

Example:

```
GET /devices?limit=100&cursor=...
```

Filtering

```
GET /devices?site=BLR&role=leaf
```

Versioning

APIs evolve.

Example:

```
/v1/devices
/v2/devices
```

or versioned media contracts.

Compatibility must be considered carefully because automation clients may be long-lived.

Error model

Avoid arbitrary errors like:

```
something went wrong
```

Prefer structured errors:

```
code:
DEVICE_UNREACHABLE

message:
Unable to contact device.

retryable:
true

correlation_id:
abc-129
```

Authentication and authorization

Authentication asks:

> Who are you?

Authorization asks:

> What are you allowed to do?

Example:

```
Engineer A:
read devices

Engineer B:
change lab networks

Network admin:
change production

Automation service:
specific machine permissions
```

---

27. Secrets and privileged access

Network automation often has highly privileged access.

Compromise of the platform can mean compromise of the entire network estate.

Secrets include:

```
device passwords
SSH private keys
API tokens
client certificates
controller credentials
```

They should not live in:

```
source code
Git
plain configuration files
logs
```

Central secret stores

Instead:

```
Worker
  |
  v
Secret manager
  |
temporary credential
  |
  v
Device
```

Rotation

Credentials must periodically change.

Automation should not require changing hard-coded values across hundreds of scripts.

Short-lived credentials

Better than a password valid for five years:

```
credential valid 15 minutes
```

Compromise then has a smaller window.

Least privilege

A telemetry collector may need:

```
read-only
```

It should not have:

```
full configuration access.
```

Similarly:

```
VLAN automation
```

should not necessarily be able to:

```
erase device configuration.
```

Auditability

Every privileged credential use should be attributable.

```
worker17
used credential role network-change
for job abc123
against router42
at 16:03
```

---

28. Audit trail

Infrastructure automation needs extremely strong auditing.

For every action answer:

```
WHO?
WHAT?
WHEN?
WHY?
WHAT WAS THE PREVIOUS STATE?
WHAT WAS THE INTENDED STATE?
WHAT SYSTEMS WERE AFFECTED?
WHO APPROVED IT?
WHAT WAS THE RESULT?
```

Example:

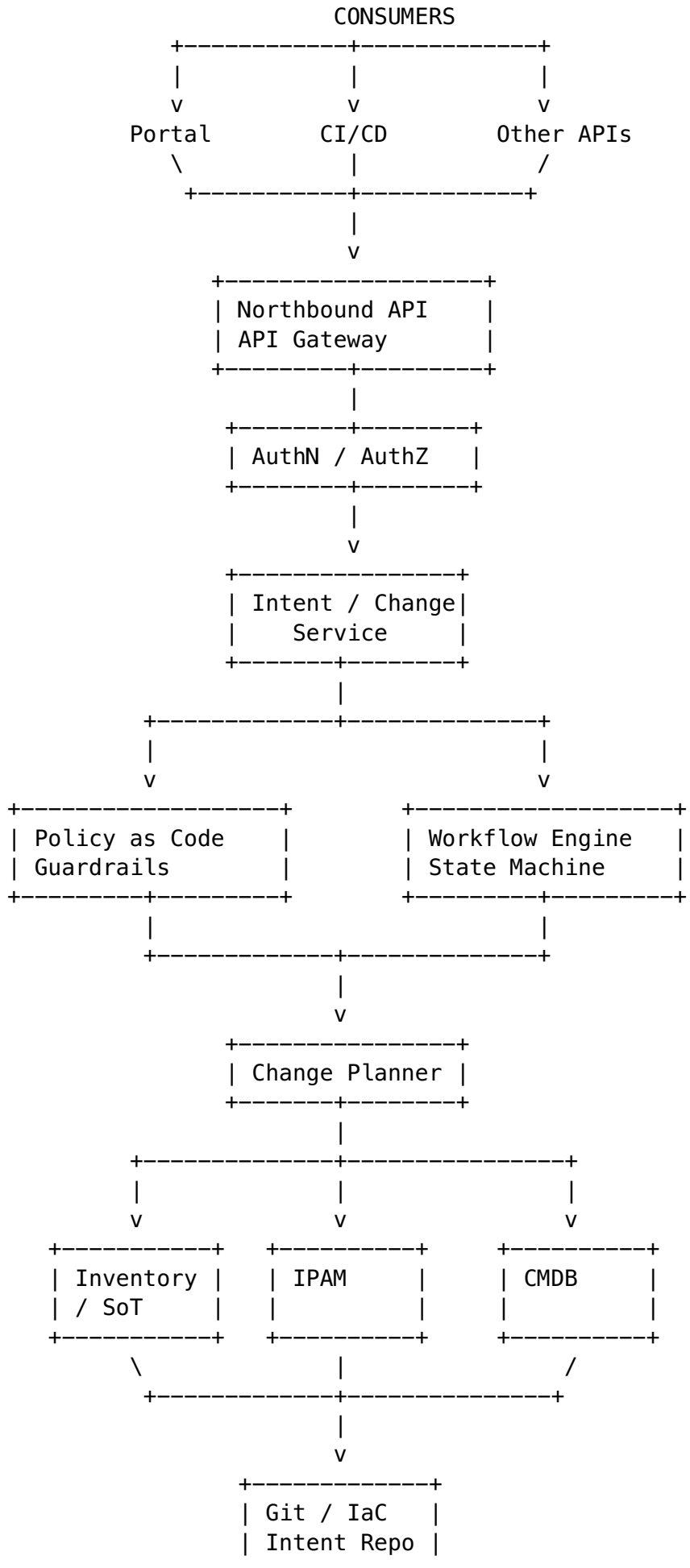
```
Change ID:
CHG-82917

Requested by:
payments-platform

Approved by:
network-production-approver

Reason:
Deploy payments DR site

Affected:
blr-leaf-21
blr-leaf-22
blr-spine-01

Previous:
VLAN 321 absent

Intended:
VLAN 321 present

Result:
Succeeded

Validation:
BGP healthy
gateway reachable
application probe healthy
```

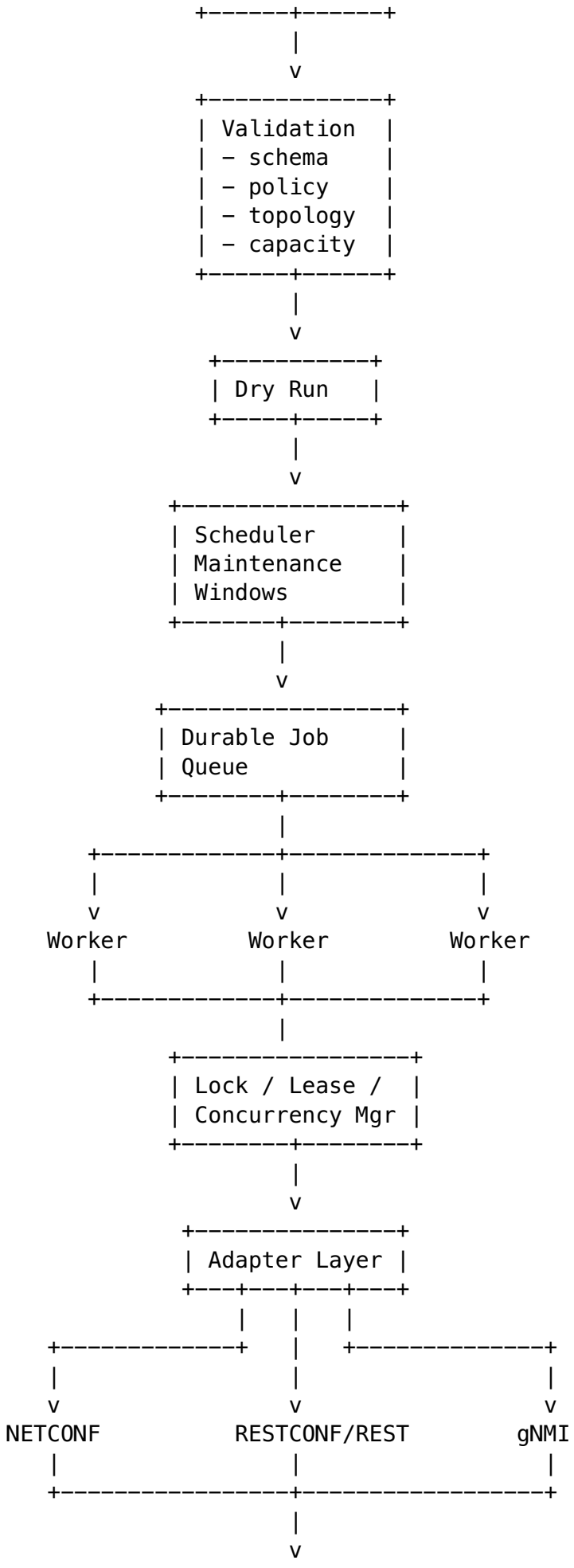
This matters for:

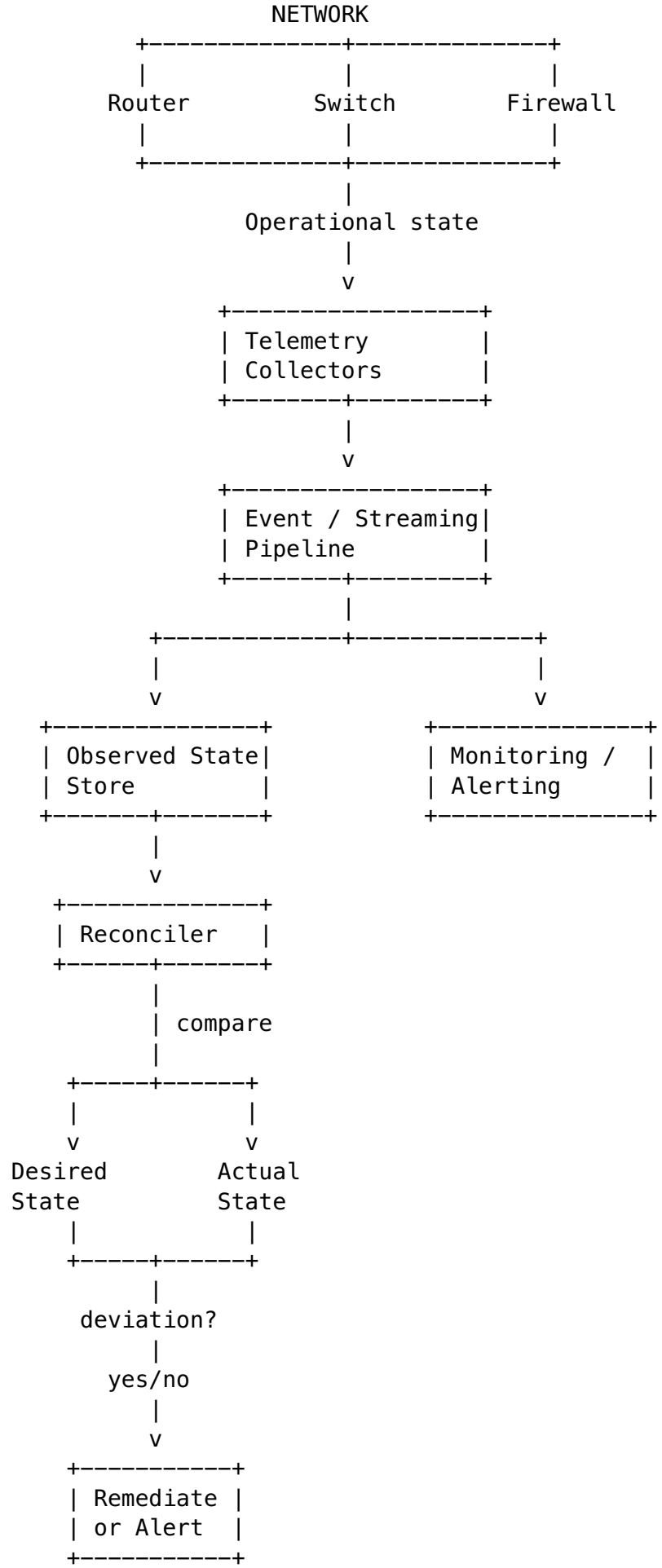
- incident investigation,
- security,
- compliance,
- debugging,
- accountability.

Putting everything together: safe enterprise network-automation platform

Here is the complete architecture.


CONSUMERS
Portal
CI/CD
Other APIs
Northbound API
API Gateway
AuthN / AuthZ
Intent / Change
Service
Policy as Code
Guardrails
Workflow Engine
State Machine
Change Planner
Inventory
/ SoT
IPAM
CMDB
Git / IaC
Intent Repo


Validation
- schema
- policy
- topology
- capacity
Dry Run
Scheduler
Maintenance
Windows
Durable Job
Queue
Worker
Worker
Worker
Lock / Lease /
Concurrency Mgr
Adapter Layer
NETCONF
RESTCONF/REST
gNMI


NETWORK
Router
Switch
Firewall
Operational state
Telemetry Collectors
Event / Streaming Pipeline
Observed State Store
Monitoring / Alerting
Reconciler
compare
Desired State
Actual State
deviation?
yes/no
Remediate or Alert

```
Every stage
     |
     v
+-------------------------------------------------+
| Audit / Change History / Metrics / Logs /       |
| Traces / Approval History / Previous State      |
+-------------------------------------------------+
```

A complete example: provisioning a network for a new application

Suppose the application team requests:

> Create a production network for the new payment reconciliation service in BLR1.

The automation platform might perform the following.

1. Receive intent

```
Application:
payment-reconciliation

Site:
BLR1

Environment:
production

Capacity:
500 endpoints
```

---

2. Authenticate

Verify the requester.

---

3. Authorize

Check whether that user or service is allowed to provision production infrastructure.

---

4. Query source of truth

Determine:

```
site
leaf switches
VRFs
network ownership
```

---

5. Allocate IP space

Ask IPAM:

```
Give me an available /23.
```

IPAM returns:

```
10.40.32.0/23
```

---

6. Apply policy

Policy may require:

```
production network
must use production VRF

must have redundant gateways

must not overlap existing ranges

must have firewall policy

changes >20 devices use progressive rollout
```

---

7. Generate desired state

```
VRF: production

VLAN:
421

Subnet:
10.40.32.0/23

Gateways:
leaf pair

Firewall:
default deny
```

---

8. Determine topology

The platform discovers:

```
12 leaf switches
2 border routers
2 firewalls
```

---

9. Validate capability

Confirm all devices support the required configuration.

---

10. Pre-check

Validate:

```
all devices reachable
redundant links healthy
routing stable
capacity available
```

---

11. Create execution plan

```
Phase 1
2 canary leafs

Phase 2
remaining leafs

Phase 3
border routing

Phase 4
firewall

Phase 5
service validation
```

---

12. Execute asynchronously

API responds:

```
job_id = network-8271
```

Workers execute the plan through:

```
NETCONF
REST APIs
gNMI
```

depending on device capabilities.

---

13. Handle retries safely

Suppose Leaf7 times out.

The operation is idempotent:

```
Ensure VLAN 421 exists.
```

The worker checks observed state and retries safely.

---

14. Post-validation

Automation verifies:

```
VLAN exists
VRF present
BGP advertisements correct
gateway reachable
firewall policy active
telemetry healthy
```

---

15. Update authoritative state

The successful network becomes part of the infrastructure source of truth.

---

16. Record audit

The system records:

```
requester
approval
intent
generated changes
devices affected
old state
new state
result
validation
```

---

17. Continue reconciliation

Later an engineer accidentally removes VLAN 421 from one switch.

Telemetry reports:

```
Observed:
VLAN missing on leaf7

Desired:
VLAN required
```

The reconciler identifies drift.

Depending on organizational policy it might:

```
automatically repair it
```

or:

```
open an incident/change request.
```

That is the transition from **automation** to **continuous infrastructure control**.

The most important Day 3 concepts to internalize

The individual technologies matter, but these architectural principles matter more.

1. Automate state, not command sequences

Prefer:

```
Ensure BGP peer exists.
```

over:

```
Execute these six CLI commands.
```

---

2. Separate desired state from observed state

```
Desired state
      !=
Observed state
```

The difference is drift.

Reconciliation closes that difference.

---

3. Assume every distributed operation can fail midway

Design assuming:

```
worker crashes
device goes offline
response disappears
dependency times out
only 78/100 devices succeed
```

Partial failure is not an edge case. It is a normal operating condition.

---

4. Idempotency makes retries safe

The combination:

```
At-least-once delivery
+
idempotency
+
deduplication
```

is often much more practical than trying to guarantee exactly-once execution.

---

5. Orchestration and automation are different

Automation answers:

> How do I configure this device?

Orchestration answers:

> How do I safely coordinate configuration of an entire infrastructure system?

---

6. Verification must validate behaviour, not merely commands

```
Command accepted
```

does not mean:

```
Network healthy.
```

---

7. Source of truth is foundational

Automation becomes dangerous when it does not know which data is authoritative.

---

8. Topology determines blast radius

Network infrastructure is interconnected.

A device cannot always be treated as an independent server.

---

9. GitOps, IaC and Kubernetes share the same core idea

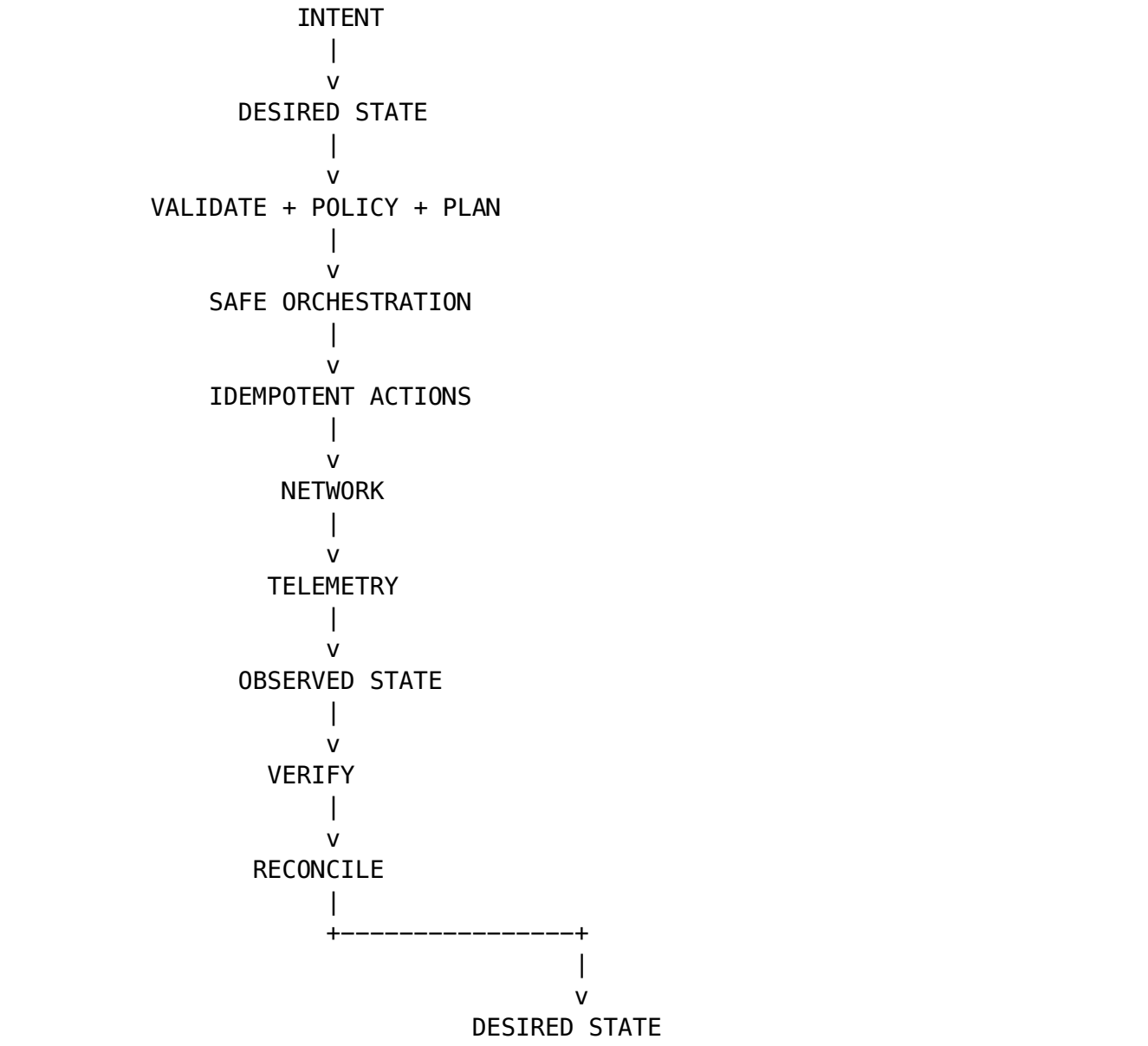
```
Declare desired state
       |
       v
Controller compares desired vs actual
       |
       v
Reconcile differences
```

This is one of the strongest conceptual bridges between **network engineering and modern platform engineering**.

---

10. Safe automation is controlled distributed change

The deepest mental model for Day 3 is therefore:



Once this loop is clear, technologies such as **NETCONF, RESTCONF, YANG, OpenConfig, gNMI, GitOps, IaC, queues, workflow engines, policy engines and streaming telemetry stop looking like unrelated tools**. They become different components of the same infrastructure-control architecture.